



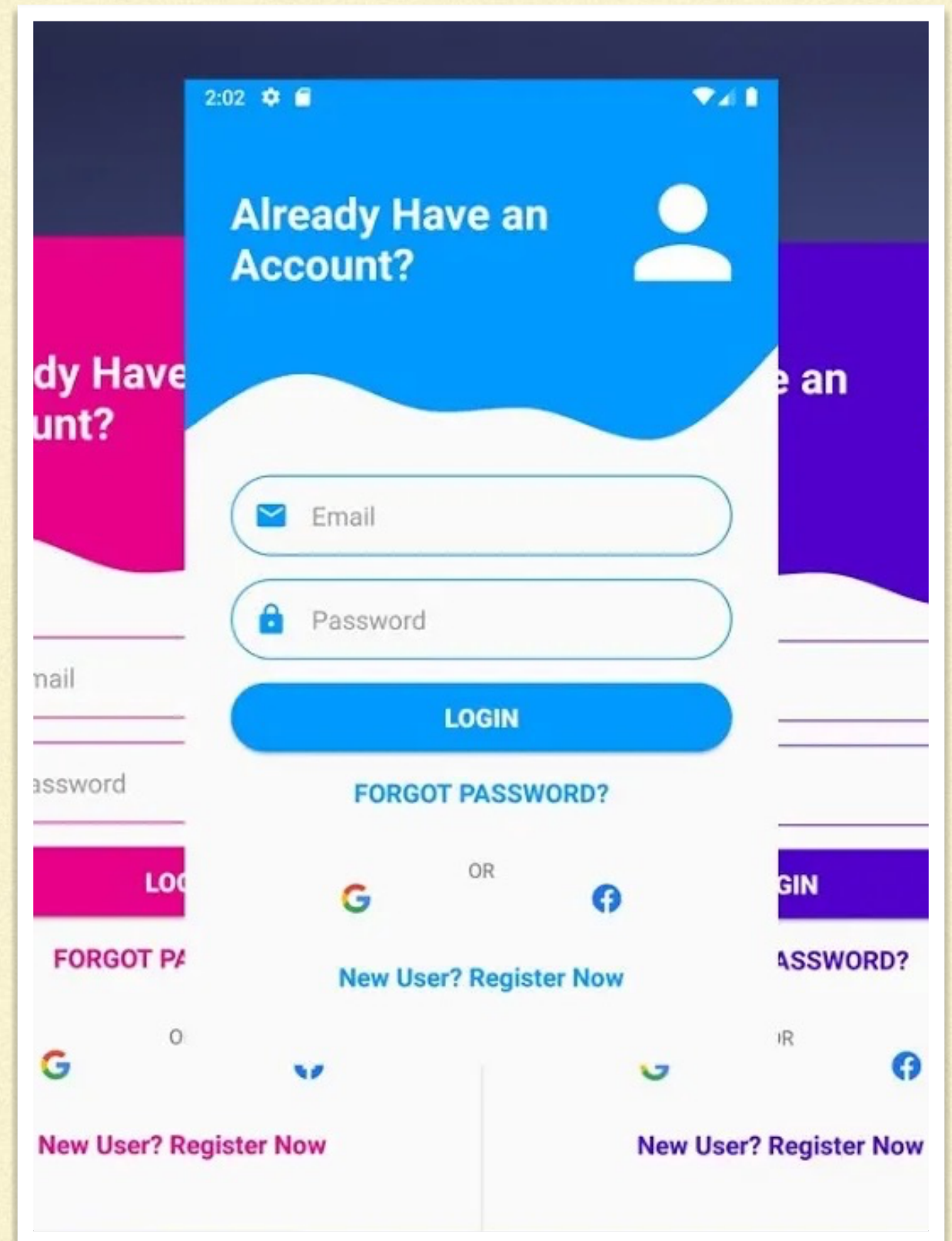
MOBILE PROGRAMMING

Mai Ngọc Châu
I

Chapter 03

USER
INTERFACE

USER
EXPERIENCE



ANDROID – USER INTERFACES USING XML LAYOUTS

All of the views in a window are arranged in a single tree.

THE VIEW CLASS

- The View class represents the basic building block for user interface components.
- A View occupies a rectangular area on the screen and is responsible for drawing and event handling.
- View is the base class for widgets, which are used to create interactive UI components (buttons, text fields, etc.).
- The ViewGroup subclass is the base class for layouts, which are invisible containers that hold other Views (or other ViewGroups) and define their layout properties.

USING VIEWS

- You can add views either **from code** or by specifying a tree of views in one or more **XML layout** files.
- Once you have created a tree of views, there are typically a few types of common operations you may wish to perform:
 1. **Set properties:** for example setting the text of a TextView. Properties that are known at build time can be set in the XML layout files.
 2. **Set focus:** The framework will handled moving focus in response to user input. To force focus to a specific view, call `requestFocus()`.
 3. **Set up listeners:** Views allow clients to set listeners that will be notified when something interesting happens to the view. For example, a Button exposes a listener to notify clients when the button is clicked.
 4. **Set visibility:** You can hide or show views using `setVisibility(int)`.

TOOL FOR EDIT XML FILE?

- Use any Text editor: you can edit xml file.
- Recommendation:
 - Android Studio: would have intelligent language support for android widgets and components.
 - IntelliJ IDEA: this IDE also support Android development.
 - ...

WHAT IS AN XML LAYOUT?

An XML-based layout is a specification of the various UI components and the relationships to each other – and to their containers – all written in XML format.



WHAT IS AN XML LAYOUT?

- Each XML file contains a **tree of elements** specifying a layout of widgets and containers that make up one View/UI.
- The attributes of the XML elements are properties, describing how a widget should look or how a container should behave.

If a Button element has an attribute value of

android:textStyle = "bold"

that means that the text appearing on the face of the button should be rendered in a boldface font style.

EXAMPLE

```
1  <?xml version="1.0" encoding="utf-8"?>
2  © <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
3     xmlns:tools="http://schemas.android.com/tools"
4     android:id="@+id/activity_main"
5     android:layout_width="match_parent"
6     android:layout_height="match_parent"
7     android:orientation="vertical"
8     tools:context=".MainActivity">
9
10     <!-- Child elements here-->
11
12 </LinearLayout>
```

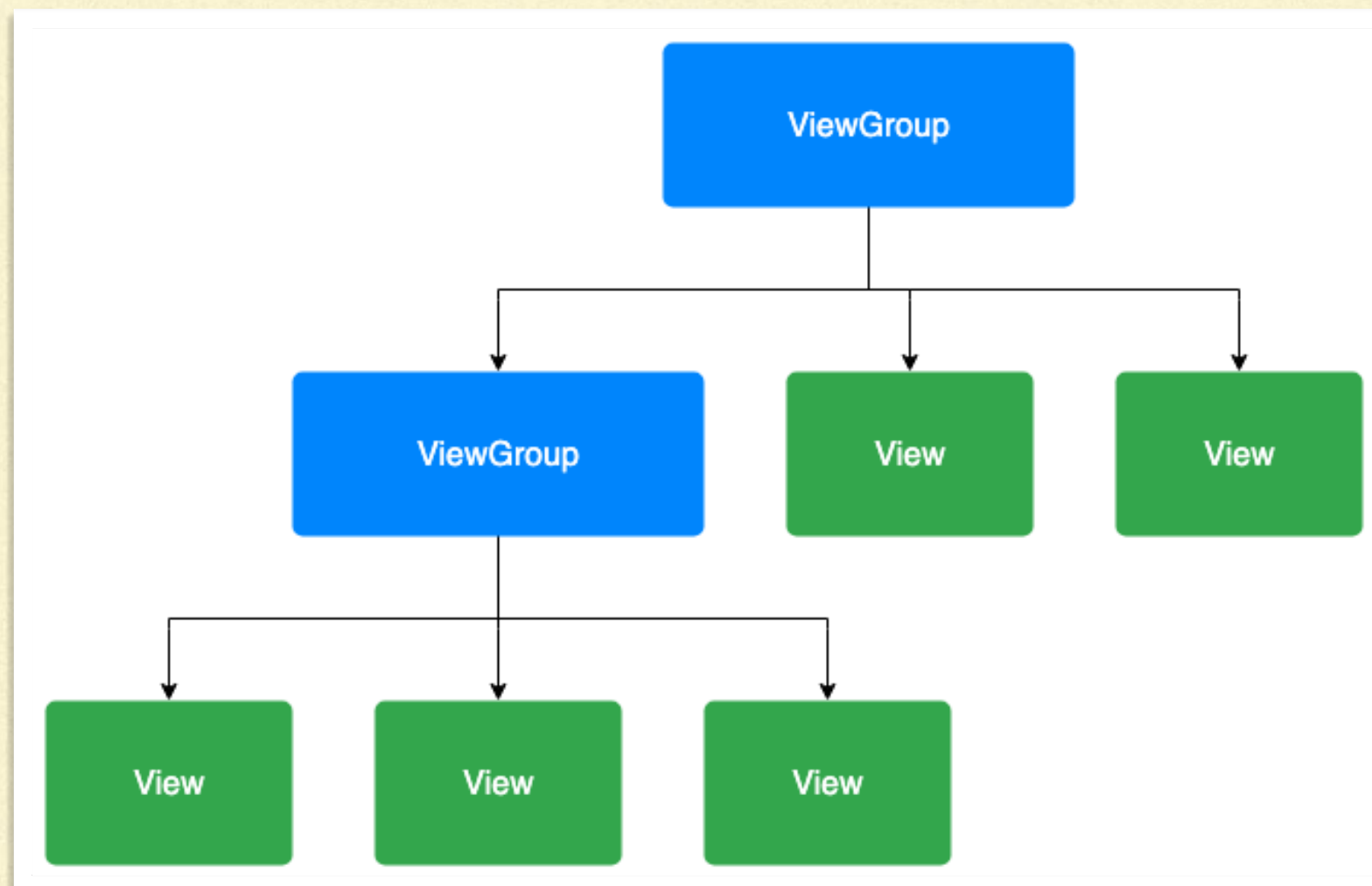
Layout Id

Attributes

Class for handle this layout

ANDROID LAYOUT

- An android app's user interface is made up of a series of View and ViewGroup elements.



VIEW

TextView
EditText
ImageView
RadioButton
Button
ImageButton
CheckBox
DatePicker
Spinner
ProgressBar and ...

VIEWGROUP

FrameLayout
WebView
ListView
GridView
LinearLayout
RelativeLayout
TableLayout
and ...

WIDGETS

<https://developer.android.com/design>



COMMON LAYOUTS - FRAMELAYOUT

- `FrameLayout` is the simplest type of layout object. It's basically a blank space on your screen that you can later fill with a single object — for example, a picture that you'll swap in and out.
- All child elements of the `FrameLayout` are pinned to the top left corner of the screen; you cannot specify a different location for a child view.
- Subsequent child views will simply be drawn over previous ones, partially or totally obscuring them (unless the newer object is transparent).

COMMON LAYOUTS - LINEARLAYOUT

- LinearLayout aligns all children in a single direction — vertically or horizontally depending on the android:orientation attribute.
- All children are stacked one after the other, so a:
 - **vertical** list will only have one child per row, no matter how wide they are.
 - **horizontal** list will only be one row high (the height of the tallest child, plus padding).
- A LinearLayout respects margins between children and the gravity (right, center, or left alignment) of each child.

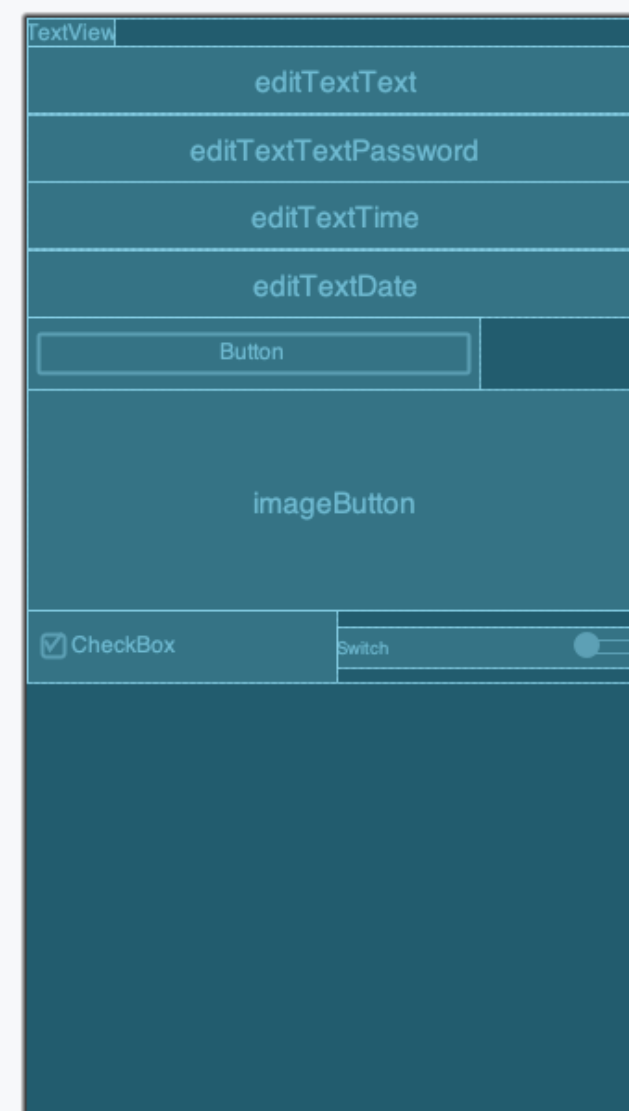
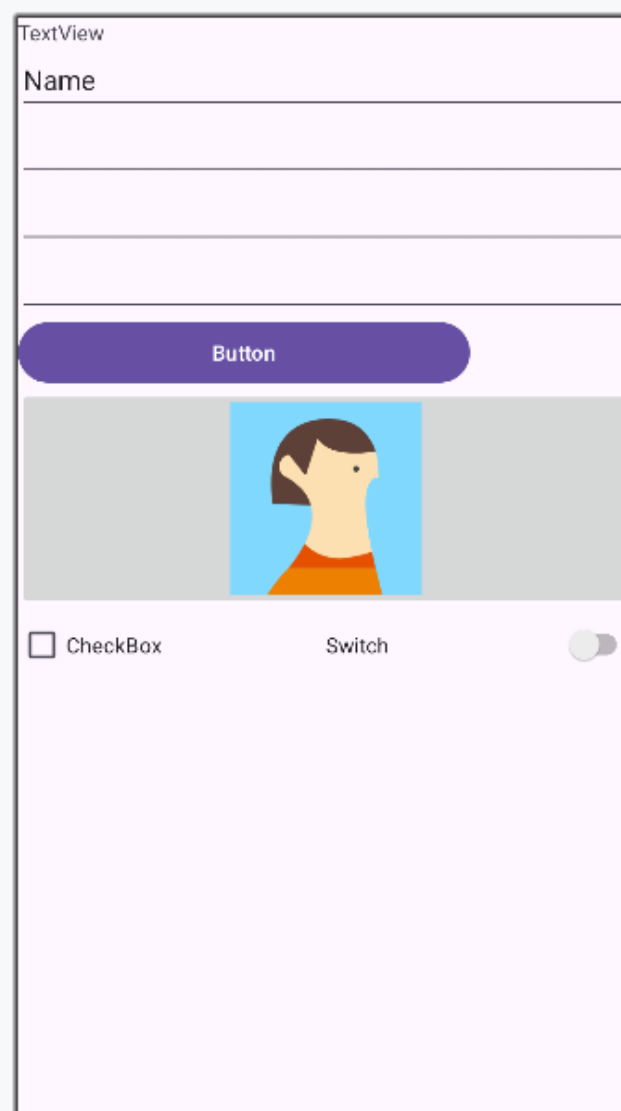
activity_main_linear.xml

```

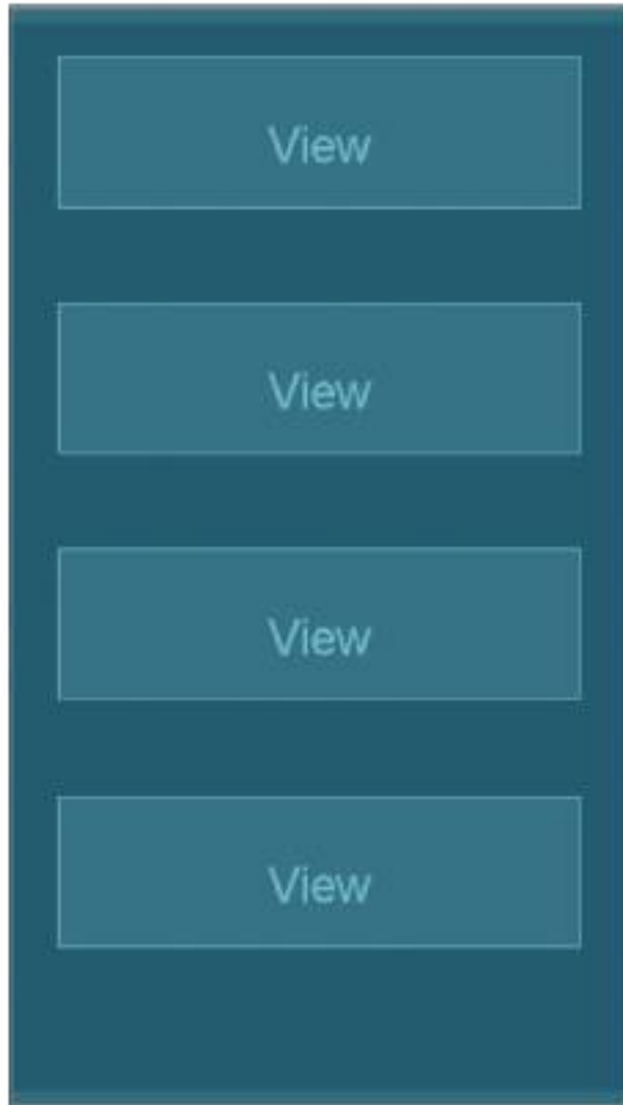
2   <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
26
27       <EditText
28           android:id="@+id/editTextTextPassword"
29           android:layout_width="match_parent"
30           android:layout_height="wrap_content"
31           android:layout_weight="1"
32           android:ems="10"
33           android:inputType="textPassword" />
34
35       <EditText
36           android:id="@+id/editTextTime"
37           android:layout_width="match_parent"
38           android:layout_height="wrap_content"
39           android:layout_weight="1"
40           android:ems="10"
41           android:inputType="time" />
42
43       <EditText
44           android:id="@+id/editTextDate"
45           android:layout_width="match_parent"
46           android:layout_height="wrap_content"
47           android:layout_weight="1"
48           android:ems="10"
49           android:inputType="date" />
50
51       <Button
52           android:id="@+id/button2"
53           android:layout_width="302dp"
54           android:layout_height="wrap_content"
55           android:layout_weight="1"
56           android:text="Button" />
57
58       <ImageButton
59           android:id="@+id/imageButton"
60           android:layout_width="match_parent"
61           android:layout_height="wrap_content"
62           android:layout_weight="1"

```

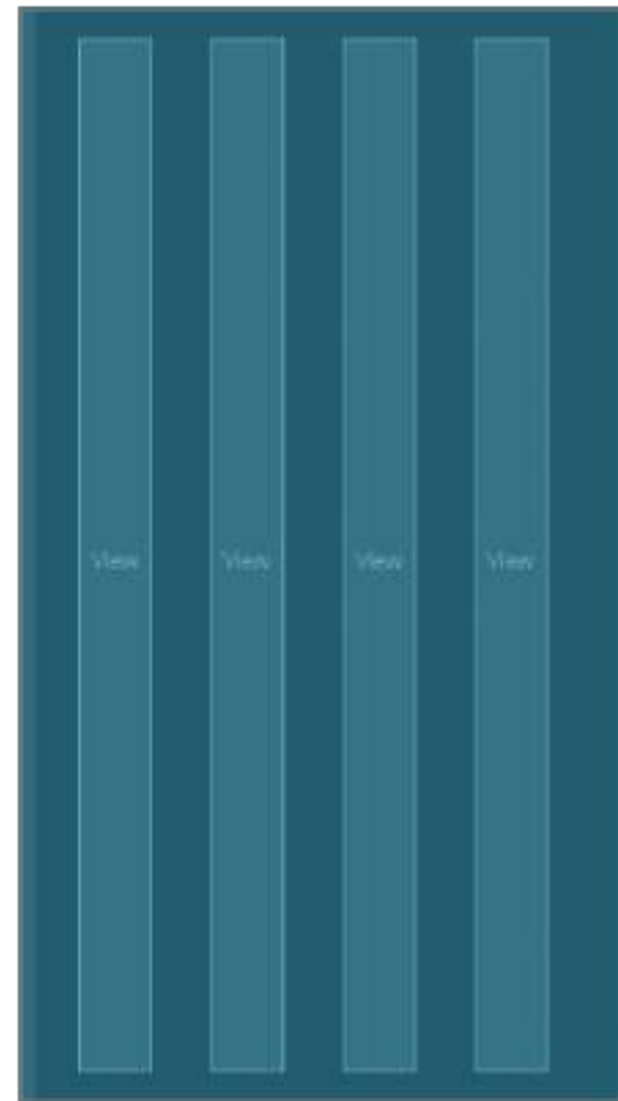
activity_main_linear.xml



LINEAR LAYOUT



`android:orientation="vertical"`



`android:orientation="horizontal"`

COMMON LAYOUTS - RELATIVELAYOUT

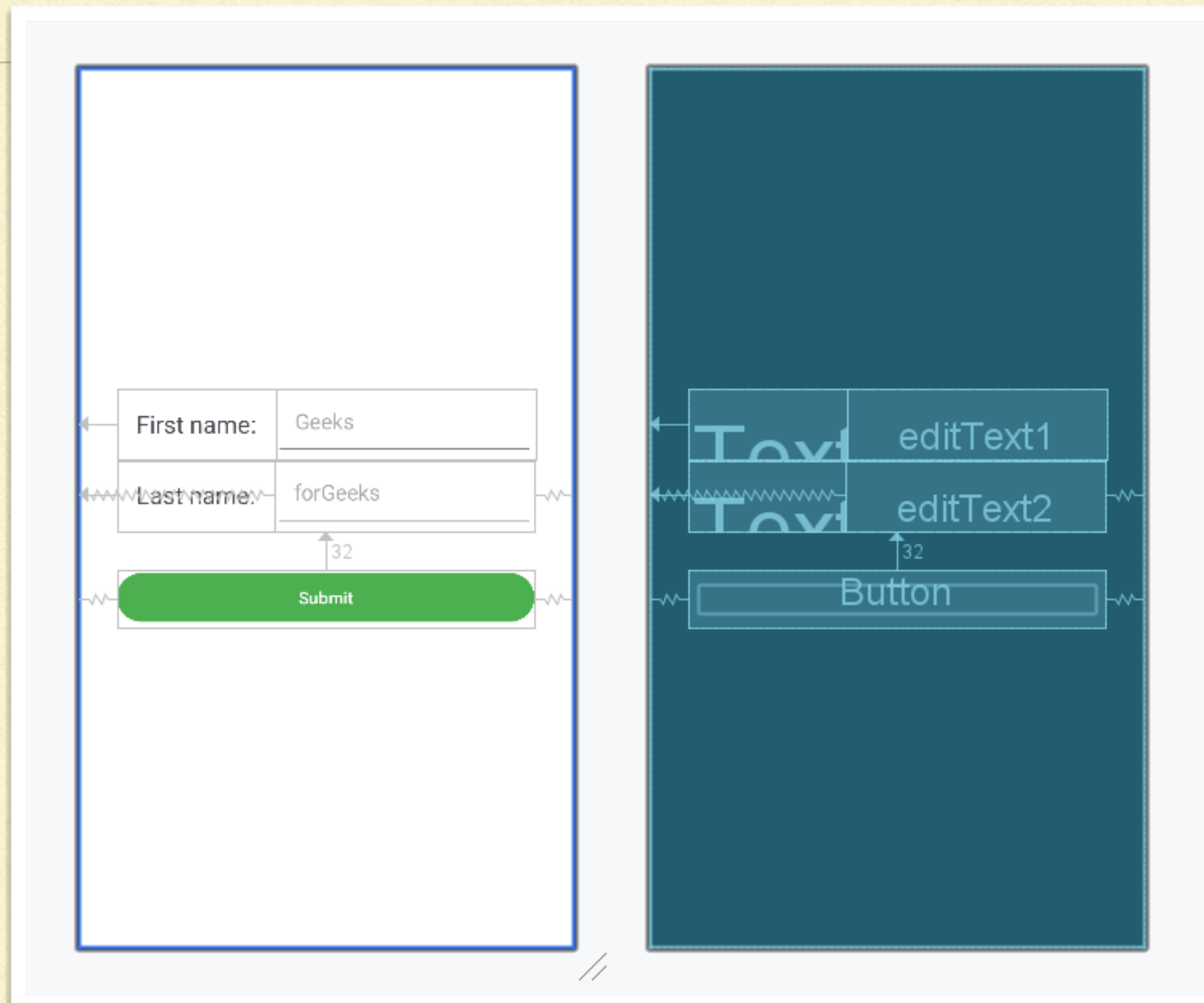
- RelativeLayout lets child views specify their position relative to the parent view or to each other (specified by ID).
- You can align two elements by right border, or make one below another, centered in the screen, centered left, and so on.
- Elements are rendered in the order given, so if the first element is centered in the screen, other elements aligning themselves to that element will be aligned relative to screen center.
- Also, because of this ordering, if using XML to specify this layout, the element that you will reference (in order to position other view objects) must be listed in the XML file before you refer to it from the other views via its reference ID.

COMMON LAYOUTS - RELATIVELAYOUT

- The defined RelativeLayout parameters are (android:layout_...):

width, below, alignTop, alignBottom, toLeftOf,	height, above alignParentTop, alignParentBottom toRightOf
padding [Bottom Left Right Top], and margin [Bottom Left Right Top].	

RELATIVE LAYOUT



COMMON LAYOUTS - TABLELAYOUT

- TableLayout positions its children into rows and columns.
- TableLayout containers do not display border lines.
- The table will have as many columns as the row with the most cells.
- A cell could be empty, but cannot span columns, as they can in HTML.
- A TableRow object defines a single row in the table.
- A row has zero or more cells, each cell is defined by any kind of other View.
- A cell may also be a ViewGroup object.

TABLE LAYOUT

ICC Ranking of Players:

Rank	Player	Team	Points
1	Virat Kohli	IND	895
2	Rohit Sharma	IND	863
3	Faf du Plessis	PAK	834
4	Steven Smith	AUS	820
5	Ross Taylor	NZ	817

TextView

TextView TextView TextView TextView

TextView TextView TextView TextView

TextView TextView TextView TextView

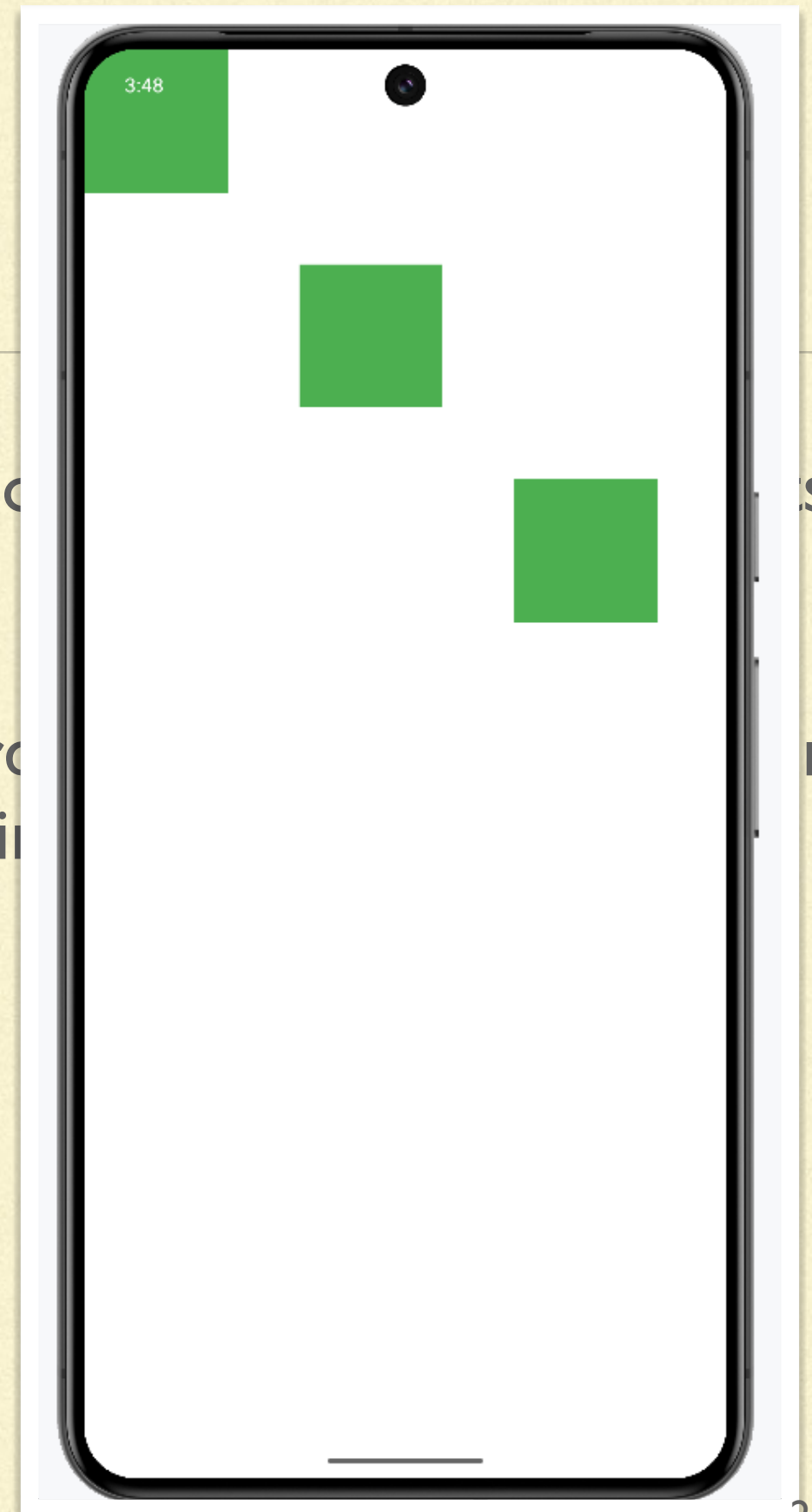
TextView TextView TextView TextView

TextView TextView TextView TextView

TextView TextView TextView TextView

COMMON LAYOUTS - ABSOLUTELAYOUT

- A layout that lets you specify exact location of children.
- Absolute layouts are less flexible and harder to use than other types of layouts without absolute positioning.



COMMON LAYOUTS - CONSTRAINTLAYOUT

- ConstraintLayout is a powerful and flexible layout manager in Android, designed to help developers create complex and responsive user interfaces with a flat view hierarchy.
- Unlike traditional layouts like LinearLayout or RelativeLayout which can lead to nested view groups, ConstraintLayout aims to minimize nesting, potentially improving UI performance.

activity_main_constraint.xml

1

<?xml version="1.0" encoding="utf-8"?>

2

<androidx.constraintlayout.widget.ConstraintLayout xmlns

3

xmlns:app="http://schemas.android.com/apk/res-auto"

4

xmlns:tools="http://schemas.android.com/tools"

5

android:id="@+id/main"

6

android:layout_width="match_parent"

7

android:layout_height="match_parent"

8

tools:context=".MainActivity">

9

10

<Button

11

android:id="@+id/button"

12

android:layout_width="234dp"

13

android:layout_height="58dp"

14

android:layout_marginTop="40dp"

15

android:text="Button"

16

app:layout_constraintEnd_toEndOf="parent"

17

app:layout_constraintHorizontal_bias=".5"

18

app:layout_constraintStart_toStartOf="parent"

19

app:layout_constraintTop_toBottomOf="@+id/editT

20

21

<TextView

22

android:id="@+id/textView3"

23

android:layout_width="wrap_content"

24

android:layout_height="wrap_content"

25

android:text="TextView"

26

tools:layout_editor_absoluteX="177dp"

27

tools:layout_editor_absoluteY="13dp" />

28

29

<EditText

30

android:id="@+id/editTextText2"

31

android:layout_width="wrap_content"

32

android:layout_height="wrap_content"

33

android:ems="10"

34

android:inputType="text"

35

android:text="Name"

36

app:layout_constraintEnd_toEndOf="@+id/textView

37

app:layout_constraintStart_toStartOf="@+id/text

38

tools:layout_editor_absoluteY="50dp" />

activity_main_constraint.xml

Pixel

35

Lesson31BasicWidgets

Default (en-us)

0dp

TextView

Name

Button



☐ CheckBox
 ☐ Switch

TextView

editTextText2

editTextTextPassword2

editTextTime2

editTextDate2

Button

imageButton2

☒ CheckBox
 ☒ Switch

24

CONSTRAINTLAYOUT - KEY CONCEPTS

- Constraints:
 - The core of ConstraintLayout involves defining constraints between UI components.
 - These constraints specify how one view is positioned relative to another view, to the parent layout, or to guidelines/barriers.
 - Constraints can be set for horizontal (left, right, start, end) and vertical (top, bottom, baseline) dimensions.

CONSTRAINTLAYOUT - KEY CONCEPTS

- Relative Positioning:
 - Views are positioned by defining relationships to other elements.
 - For example, you can constrain the top of one TextView to the bottom of another ImageView.
- Chains:
 - Chains allow you to manage the spacing and distribution of a group of views in a single dimension (horizontal or vertical).
 - Different chain styles (e.g., spread, spread_inside, packed) control how the space is distributed.

CONSTRAINTLAYOUT

Recommended to use for model android app. However:

Since we will focus on every aspect of Android development
We will try not to dig too deep into complicated layout and UI design

RELATIVELAYOUT VS CONSTRAINTLAYOUT

Đặc điểm	RelativeLayout	ConstraintLayout
Cách định vị	Dựa trên mối quan hệ tương đối với các view khác hoặc với layout cha.	Dựa trên các ràng buộc (constraints) giữa các view.
Hiệu suất	Kém hơn, đặc biệt với layout phức tạp và lồng nhau.	Tốt hơn, giảm nhu cầu lồng layout.
Độ linh hoạt	Giới hạn, khó tạo layout phức tạp.	Rất linh hoạt, có thể tạo hầu hết mọi loại layout.
Công cụ hỗ trợ	Hỗ trợ tốt trong Android Studio nhưng không mạnh bằng ConstraintLayout.	Có công cụ trực quan Design Editor mạnh mẽ.
Khuyến cáo sử dụng	Các layout đơn giản, khi cần sắp xếp nhanh các view theo mối quan hệ tương đối.	Được khuyến khích cho hầu hết các layout mới, từ đơn giản đến phức tạp, để tối ưu hiệu suất và dễ dàng bảo trì.

AbsListView	EditText	OverScroller	SlidingDrawer
AbsListView.LayoutParams	ExpandableListView	PopupMenu	Space
AbsoluteLayout	ExpandableListView.ExpandableListContextMenuInfo	PopupWindow	Spinner
AbsoluteLayout.LayoutParams	Filter	ProgressBar	StackView
AbsSeekBar	Filter.FilterResults	QuickContactBadge	Switch
AbsSpinner	FrameLayout	RadioButton	TabHost
ActionMenuView	FrameLayout.LayoutParams	RadioGroup	TabHost.TabSpec
ActionMenuView.LayoutParams	Gallery	RadioGroup.LayoutParams	TableLayout
AdapterView<T extends Adapter>	Gallery.LayoutParams	RatingBar	TableLayout.LayoutParams
AdapterView.AdapterContextMenuInfo	GridLayout	RelativeLayout	TableRow
AdapterViewAnimator	GridLayout.Alignment	RelativeLayout.LayoutParams	TableRow.LayoutParams
AdapterViewFlipper	GridLayout.LayoutParams	RemoteViews	TabWidget
AlphabetIndexer	GridLayout.Spec	RemoteViews.DrawInstructions	TextClock
AnalogClock	GridView	RemoteViews.DrawInstructions.Builder	TextSwitcher
ArrayAdapter<T>	HeaderViewListAdapter	RemoteViews.RemoteCollectionItems	TextView
AutoCompleteTextView	HorizontalScrollView	RemoteViews.RemoteCollectionItems.Builder	TextView.SavedState
BaseAdapter	ImageButton	RemoteViews.RemoteResponse	TimePicker
BaseExpandableListAdapter	ImageSwitcher	RemoteViews.RemoteViewOutlineProvider	Toast
Button	ImageView	RemoteViewsService	Toast.Callback
CalendarView	LinearLayout	ResourceCursorAdapter	ToggleButton
CheckBox	LinearLayout.LayoutParams	ResourceCursorTreeAdapter	Toolbar
CheckedTextView	ListPopupWindow	Scroller	Toolbar.LayoutParams
Chronometer	ListView	ScrollView	TwoLineListItem
CompoundButton	ListView.FixedViewInfo	SearchView	VideoView
CursorAdapter	Magnifier	SeekBar	ViewAnimator
CursorTreeAdapter	Magnifier.Builder	ShareActionProvider	ViewFlipper
DatePicker	MediaController	SimpleAdapter	ViewSwitcher
DialerFilter	MultiAutoCompleteTextView	SimpleCursorAdapter	ZoomButton
DigitalClock	MultiAutoCompleteTextView.CommaTokenizer	SimpleCursorTreeAdapter	ZoomButtonsController
EdgeEffect	NumberPicker	SimpleExpandableListAdapter	ZoomControls

Basic UI Controls:

- **Buttons:** `Button`, `ImageButton`, `FloatingActionButton`
- **Text Input:** `EditText`, `AutoCompleteTextView`, `TextInputLayout`
- **Selection Controls:** `CheckBox`, `RadioButton`, `RadioGroup`, `Switch`, `ToggleButton`
- **Progress Indicators:** `ProgressBar`, `SeekBar`, `RatingBar`
- **Display Elements:** `TextView`, `ImageView`

Layout Containers:

- **Linear Layout:** `LinearLayout` (arranges views in a single row or column)
- **Relative Layout:** `RelativeLayout` (positions views relative to each other or the parent)
- **Constraint Layout:** `ConstraintLayout` (flexible and powerful layout manager)
- **Frame Layout:** `FrameLayout` (overlays views on top of each other)
- **Grid Layout:** `GridLayout` (arranges views in a grid)
- **Table Layout:** `TableLayout` (arranges views in a table)

Collection Views (for displaying lists and grids of data):

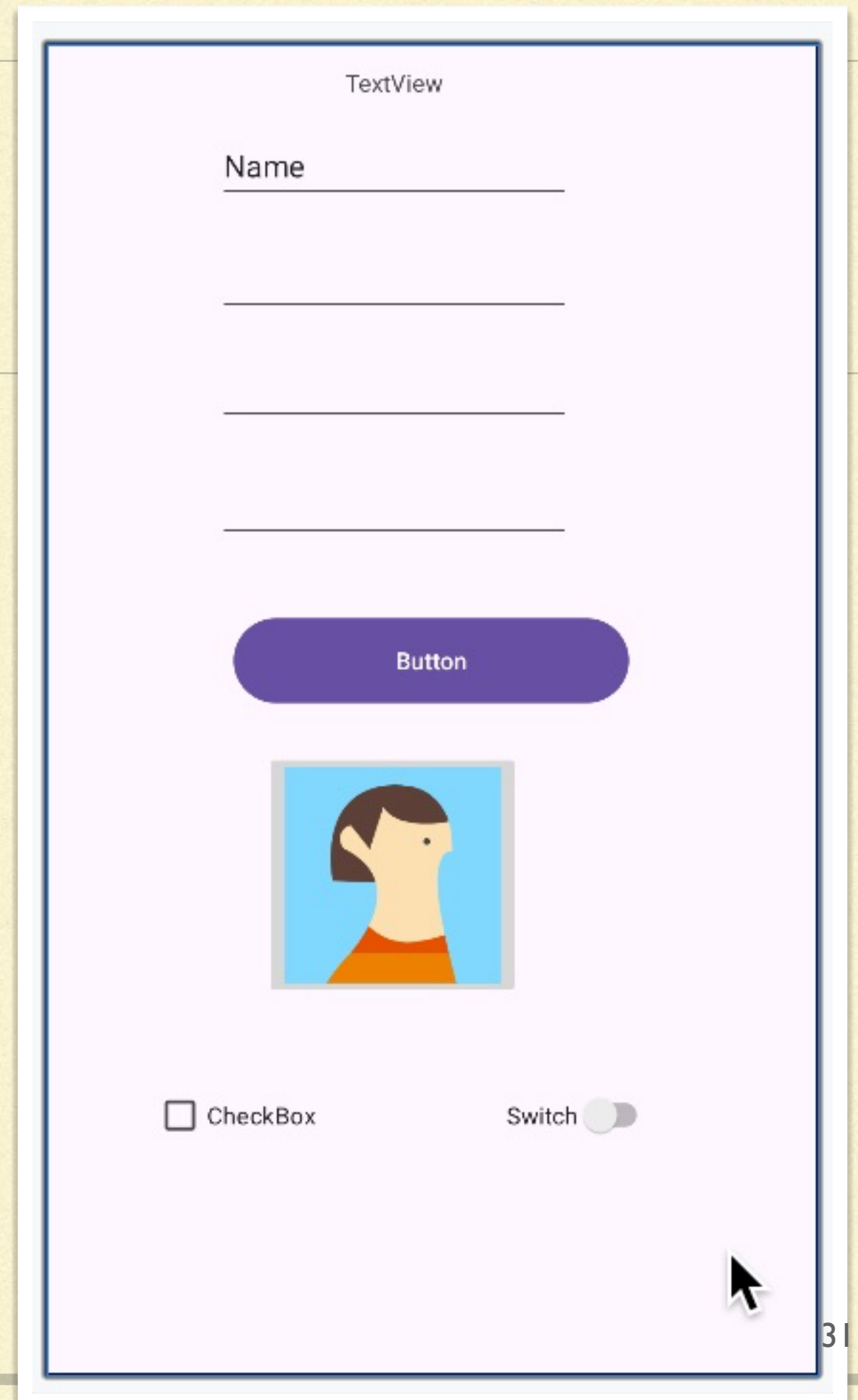
- **List View:** `ListView` (for displaying scrollable lists)
- **Recycler View:** `RecyclerView` (modern and efficient alternative to `ListView`)
- **Grid View:** `GridView` (for displaying items in a scrollable grid)
- **Spinner:** `Spinner` (a dropdown list for selecting an item)

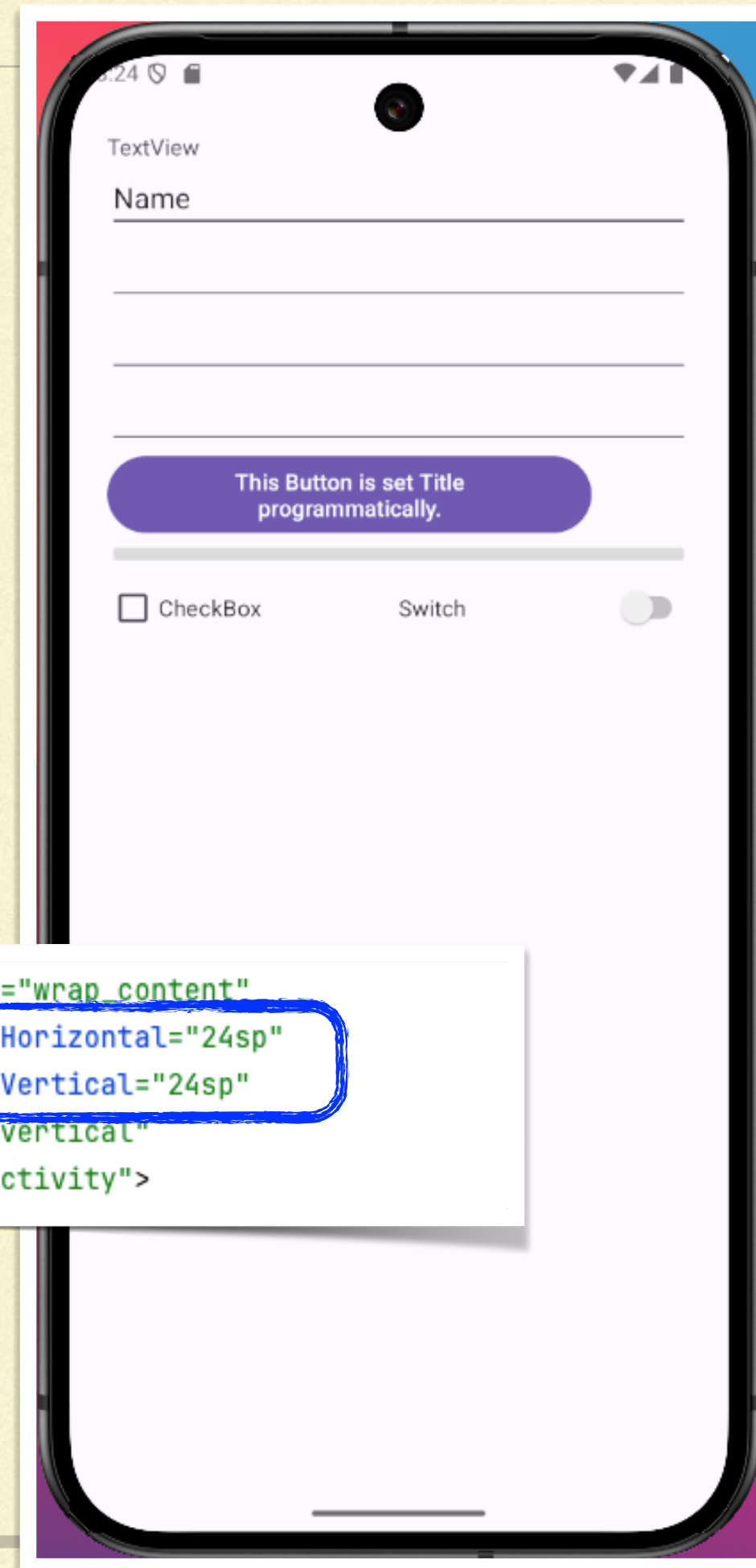
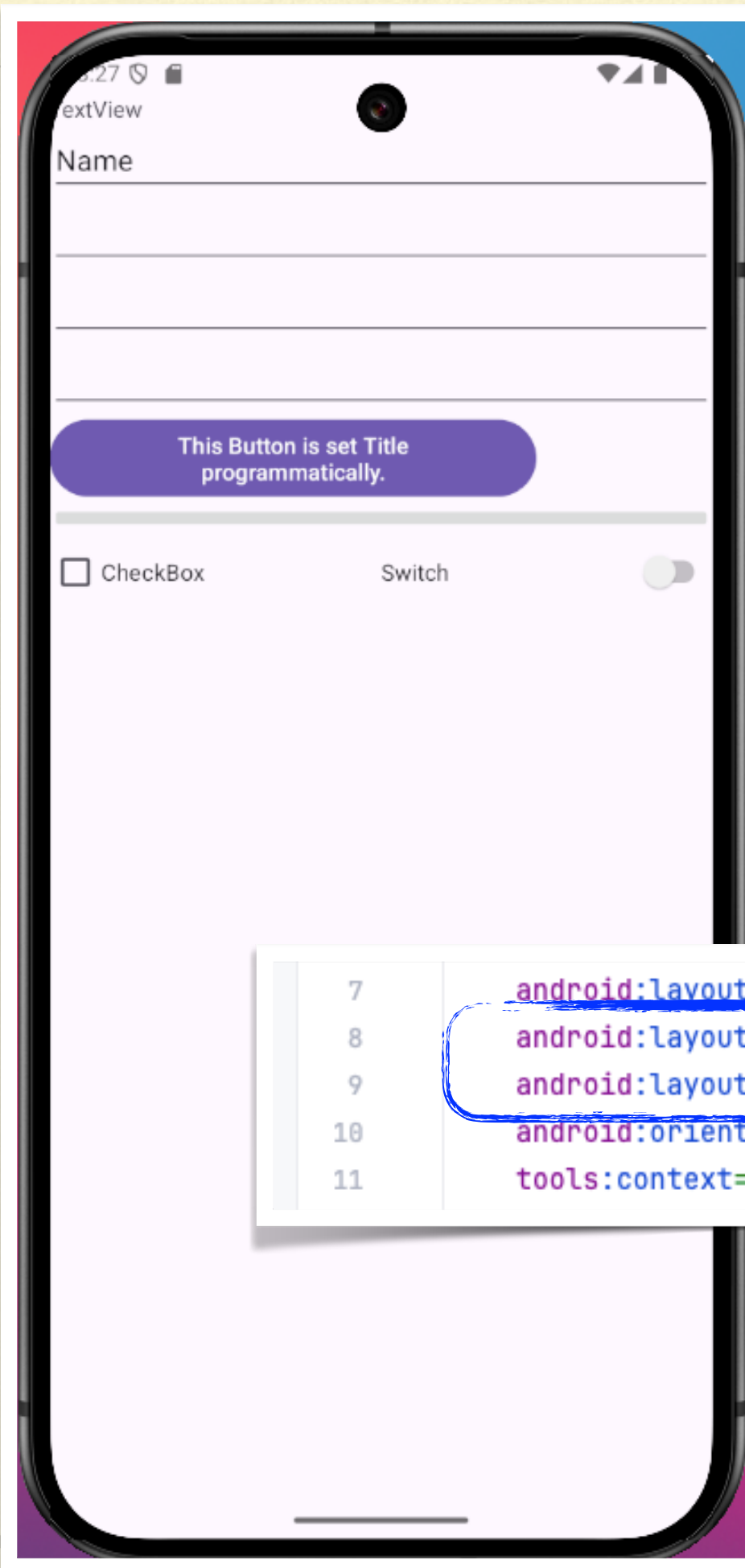
Specialized Widgets:

- **Date and Time Pickers:** `DatePicker`, `TimePicker`
- **Web View:** `WebView` (for displaying web content within the app)
- **Video View:** `VideoView` (for playing video)
- **Search View:** `SearchView` (for implementing search functionality)
- **Card View:** `CardView` (a container with a rounded corner and shadow effect)
- **Snackbar:** `Snackbar` (a lightweight feedback message)
- **Bottom Navigation View:** `BottomNavigationView` (for navigation at the bottom of the screen)
- **Drawer Layout:** `DrawerLayout` (for implementing a navigation drawer)

DEMO BASIC WIDGET

- Demonstration for:
 - Linear layout vs Constraint layout
 - Show some basic widget and how it being place on the layout.





```
7      android:layout_height="wrap_content"  
8      android:layout_marginHorizontal="24sp"  
9      android:layout_marginVertical="24sp"  
10     android:orientation="vertical"  
11     tools:context=".MainActivity">
```


ATTACHING LAYOUTS TO JAVA CODE

- Assume the UI in `res/layout/main.xml` has been created. This layout could be called by an application using the statement

```
setContentView(R.layout.main);
```

- Individual widgets, such as `myButton` could be accessed by the application using the statement `findViewById(...)` as in

```
Button btn = (Button) findViewById(R.id.myButton);
```

- Where `R` is a class automatically generated to keep track of resources available to the application. In particular `R.id...` is the collection of widgets defined in the XML layout.

ATTACHING LAYOUTS TO JAVA CODE

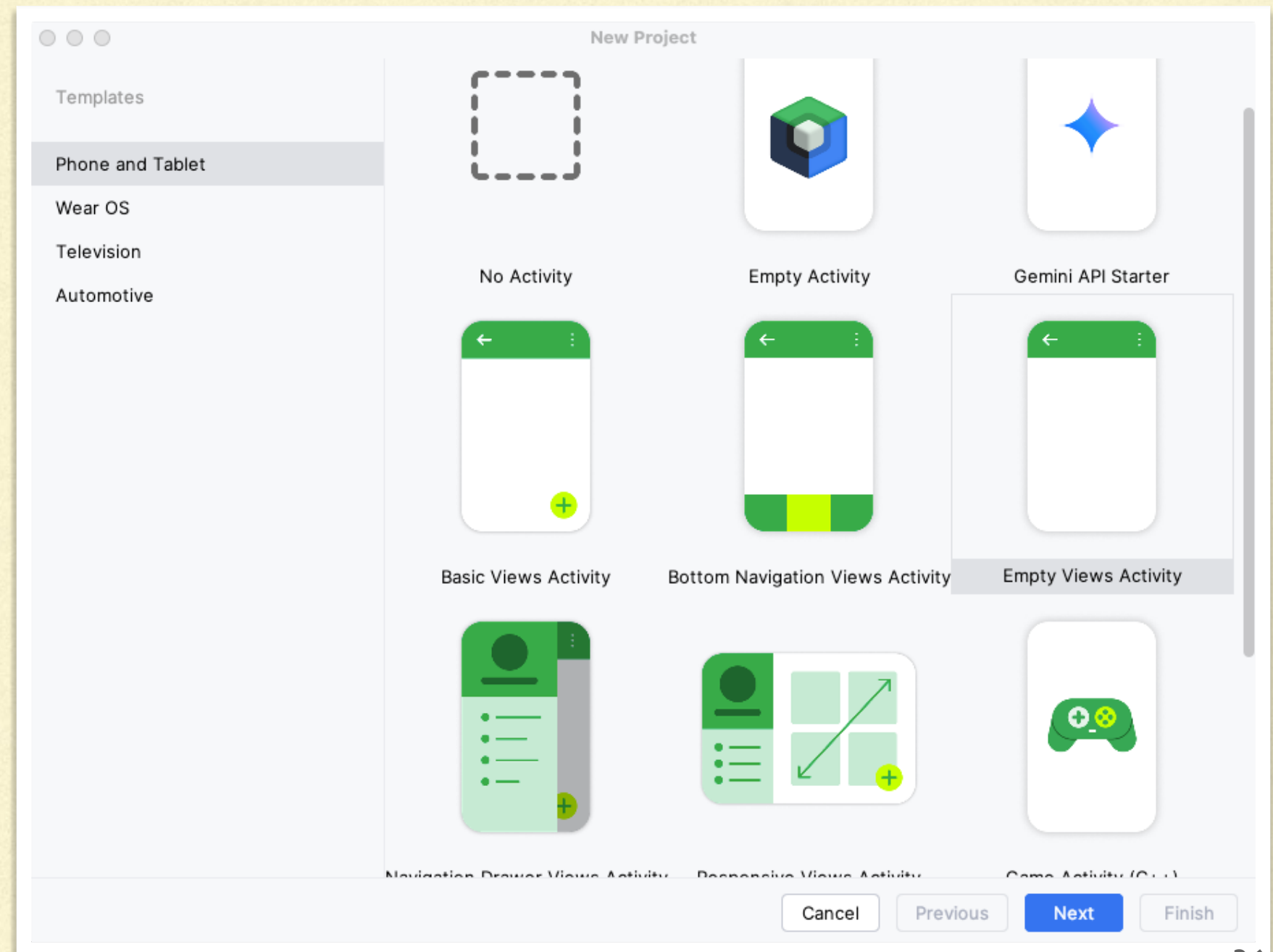
- Attaching Listeners to the Widgets
- The button of our example could now be used, for instance a listener for the click event could be written as:

```
btn.setOnClickListener(new OnClickListener() {  
    @Override  
    public void onClick(View v) {  
        btn.setText(new Date().toString())  
    }  
});
```


TIME - TO - DEMO

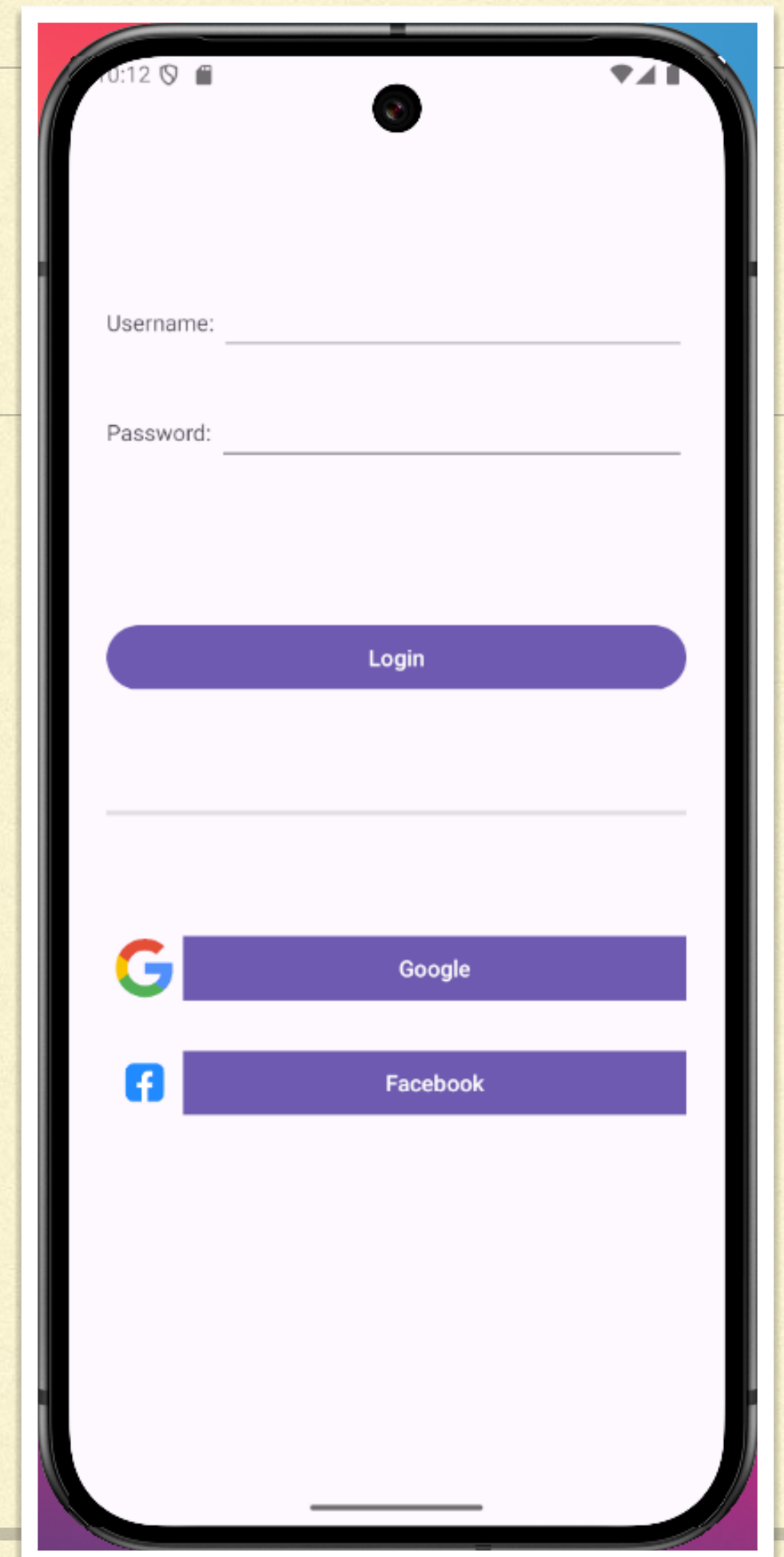
BASIC WIDGET: LOGIN / CHECKBOX

1. Add new Empty Views Activity project.
2. Fix main layout.
3. Fix Java code to handle main button (just simple enough for demo).



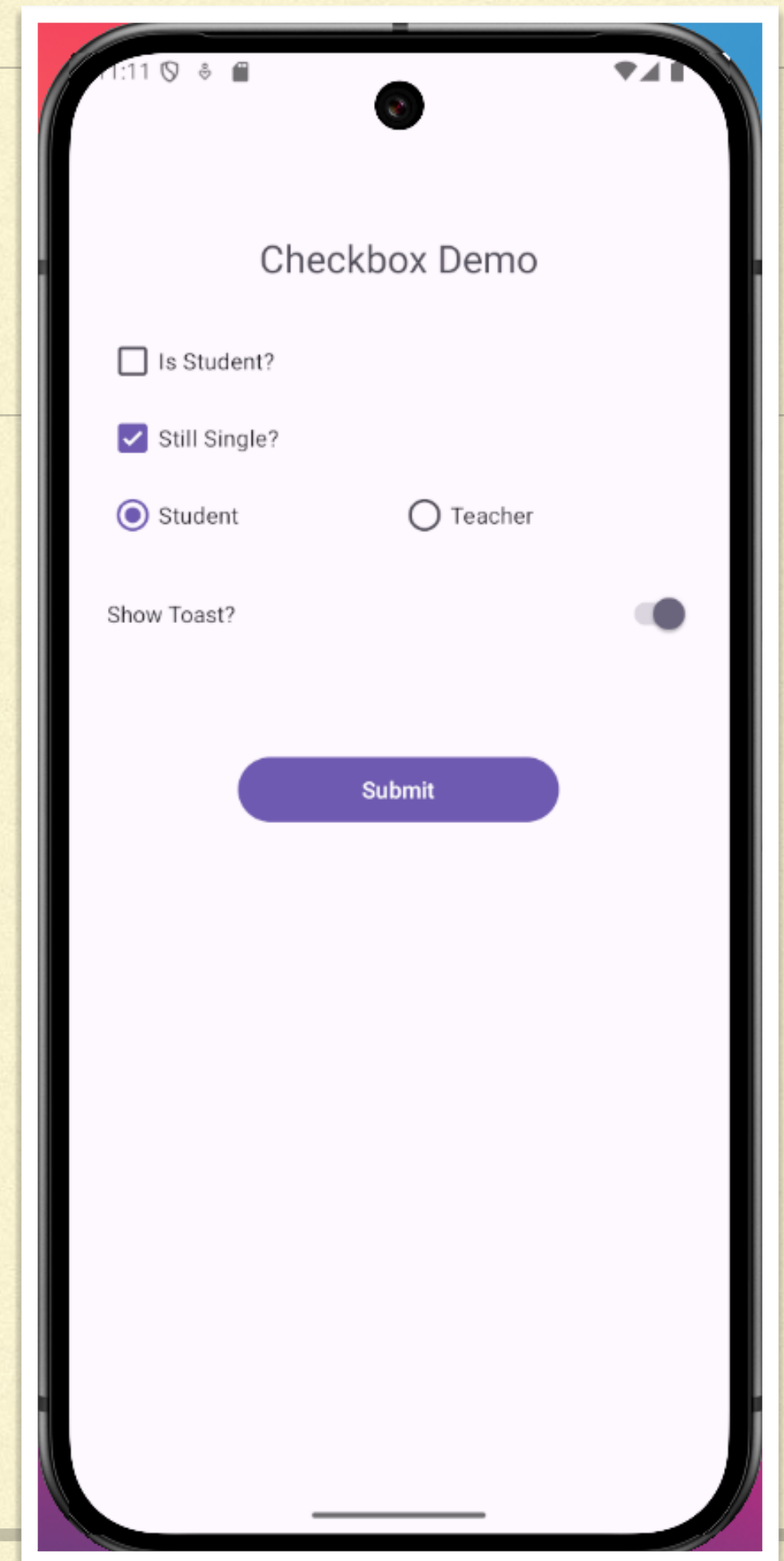
BASIC WIDGET LOGIN LAYOUT

- Demonstration for:
 - Linear layout
 - TextView/Label
 - TextInput/EditText
 - Buttons



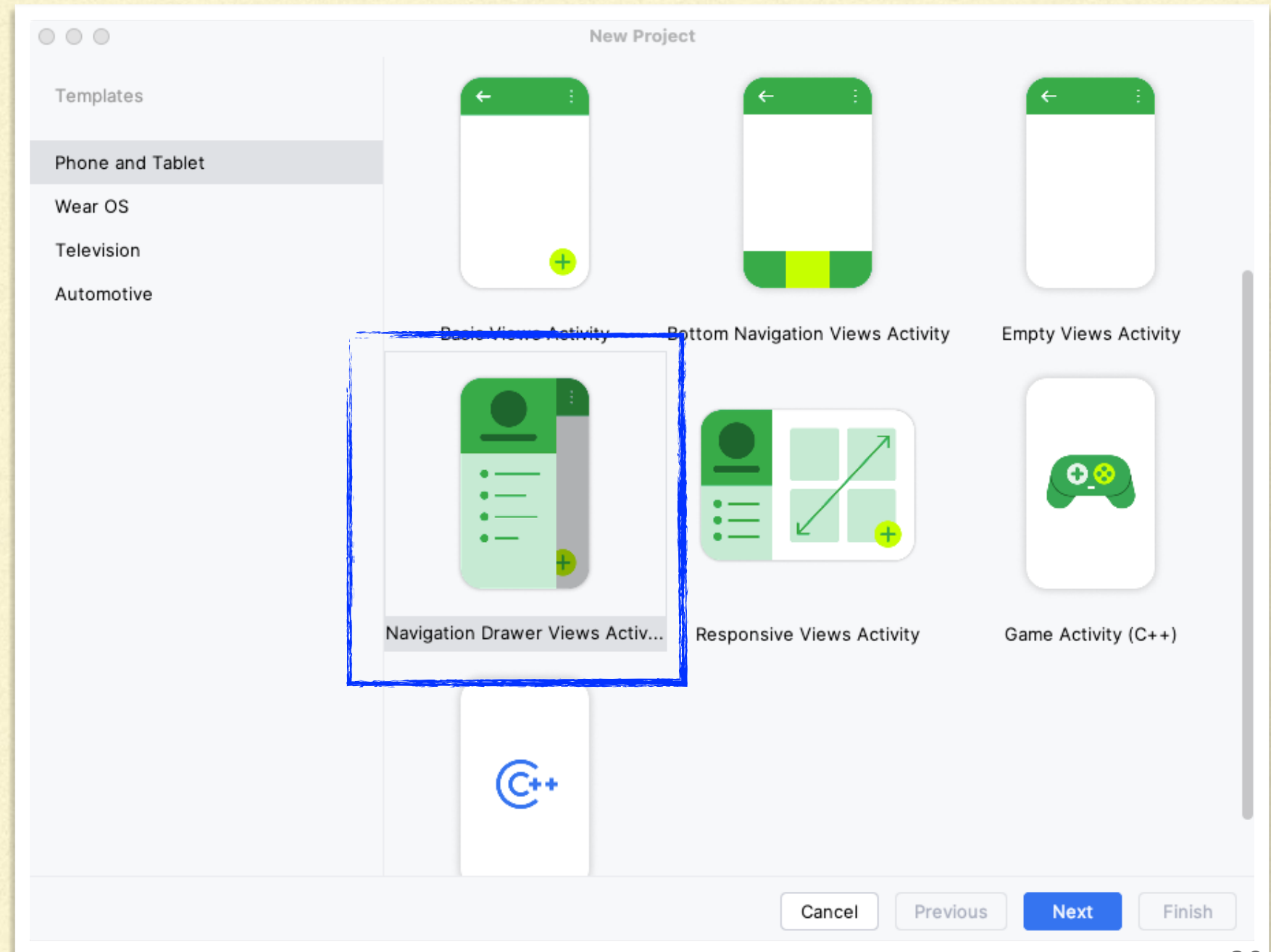
BASIC WIDGET CHECKBOX

- Demonstration for choosing from options:
 - Checkbox
 - RadioButtons
 - Switch



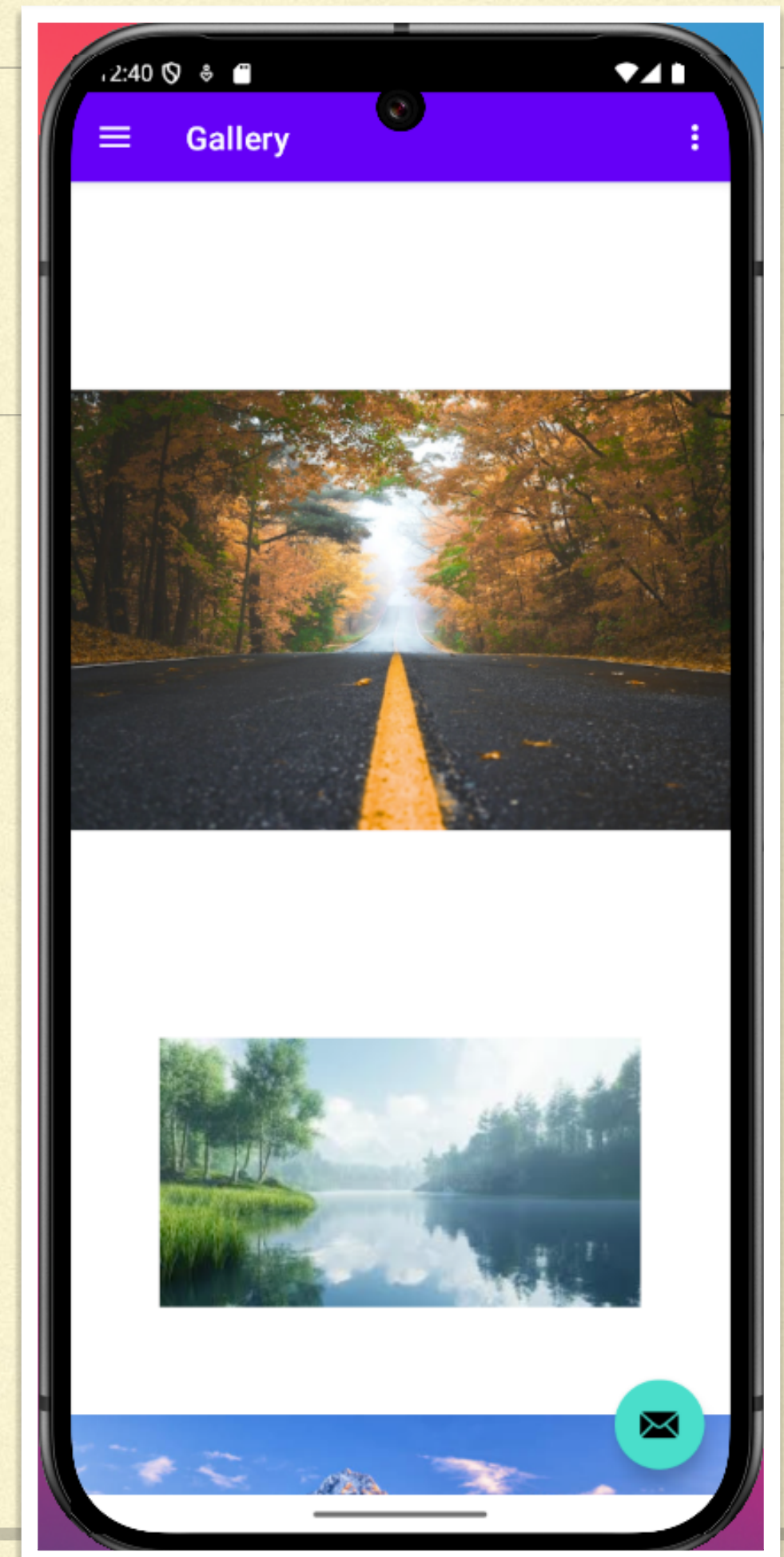
BASIC WIDGET IMAGES

1. Add new modern Project, with Sidebar Menu.
2. Add images for display.
3. Fix gallery layout.
4. Fix Java code, since we change/remove some widgets.



BASIC WIDGET IMAGES

- Demonstration for:
 - ImageView
 - ImageButton



MainActivity.java

activity_main_drawer.xml

fragment_gallery.xml

GalleryFragment.java

fragment_gallery.xml

Pixel

35

Lesson34BasicImages

Default (en-u

1

<?xml version="1.0" encoding="

2

<ScrollView xmlns:android="http://schemas

3

xmlns:app="http://schemas.android.co

4

xmlns:tools="http://schemas.android.

5

android:layout_width="match_parent"

6

android:layout_height="match_parent"

7

android:contentDescription="Image li

8

9

<LinearLayout

10

android:layout_width="match_pare

11

android:layout_height="match_par

12

android:orientation="vertical"

13

tools:context=".vi.gallery.Galle

14

15

<ImageView

16

android:id="@+id/imageView2"

17

android:layout_width="match_

18

android:layout_height="wrap_

19

android:src="@drawable/image

20

21

<ImageView

22

android:id="@+id/imageView3"

23

android:layout_width="match_

24

android:layout_height="wrap_

25

android:src="@drawable/image

26

27

<ImageView

28

android:id="@+id/imageView4"

29

android:layout_width="match_

30

android:layout_height="wrap_

31

android:src="@drawable/image

32

</LinearLayout>

33

</ScrollView>

34

35

Palette

Component Tree

End of Chapter 03
THANK YOU!

